UNIVERSIDAD TECNOLOGICA NACIONAL

Tecnicatura Universitaria en Programacion

Materia: Programacion 1

Trabajo Practico Integrador

Gestion de Datos de Paises en Python: filtros, ordenamientos y estadisticas

Integrantes	Bautista Gil
	Agustin Tomadin
Comision	2 - Tarde
Docente	Ariel Enferrel
Fecha de entrega	27/10/2026

Este documento lo armamos con ayuda de Claude (IA de Anthropic).

Indice

1. Introduccion **3**

2. Marco Teorico **3**

2.1 Listas . . . 3

2.2 Diccionarios . . . 3

2.3 Funciones . . . 3

2.4 Condicionales y repetitivas . . . 3

2.5 Ordenamientos . . . 3

2.6 Estadisticas basicas . . . 3

2.7 Archivos CSV . . . 3

3. Decisiones Tecnicas y Arquitectura **4**

3.1 Diagrama de flujo . . . 4

3.2 Capturas de ejecucion . . . 5

4. Dificultades y Conclusiones **8**

4.1 Dificultades . . . 8

4.2 Conclusiones . . . 8

5. Links del proyecto **8**

1. Introduccion

Hicimos un programa en Python para manejar datos de paises: nombre, poblacion, superficie y continente. Los datos salen de un archivo CSV y se usan desde un menu en la consola, donde se pueden agregar, actualizar, buscar, filtrar y ordenar paises, y ver algunas estadisticas. Cuando se agrega o se cambia algo, se guarda en el CSV.

2. Marco Teorico

2.1 Listas

Una lista guarda varios elementos en una sola variable, en orden. Nosotros guardamos todos los paises en una lista y le agregamos paises nuevos con `append()`.

2.2 Diccionarios

Un diccionario guarda datos en pares clave: valor. Cada pais es un diccionario, asi que para ver su poblacion ponemos `pais["poblacion"]`, que se entiende mucho mas que usar una posicion.

```
{"nombre": "Argentina", "poblacion": 45376763,
 "superficie": 2780400, "continente": "America"}
```

2.3 Funciones

Una funcion es un pedazo de codigo con nombre que hace una sola cosa. Dividimos el programa en funciones chicas (pedir un numero, buscar un pais, mostrar la lista...) y las reusamos donde hacen falta.

2.4 Condicionales y repetitivas

Con `if/elif/else` el programa decide que hacer segun la opcion que elige el usuario. Con `for` recorremos los paises, y el menu es un `while` que se repite hasta que el usuario elige salir.

2.5 Ordenamientos

Para ordenar usamos el metodo burbuja, hecho a mano: se comparan los paises de a dos y, si estan al reves, se intercambian. Esto se repite hasta que queda todo ordenado.

2.6 Estadisticas basicas

Sacamos el pais con mas y menos poblacion, el promedio de poblacion y de superficie (la suma dividida por la cantidad) y cuantos paises hay por continente, contandolos en un diccionario.

2.7 Archivos CSV

Un CSV es un archivo de texto donde cada linea es un registro y los datos van separados por comas. Lo leemos linea por linea, cortamos cada una con `split(",")` y pasamos los numeros de texto a entero con `int()`. Si una linea esta mal, el programa avisa y la saltea.

3. Decisiones Tecnicas y Arquitectura

- Usamos una **lista de diccionarios**: la lista tiene todos los paises y cada diccionario es un pais.
- Hicimos el **ordenamiento a mano** (burbuja) en vez de usar `sorted()`, para entender como funciona.
- El CSV se **guarda cada vez que hay un cambio**, asi no se pierde nada si se cierra el programa.
- Sacamos las **tildes** de todo lo que entra, para que "America" con o sin tilde sea lo mismo.
- Si algun dato esta mal, el programa **muestra el error y vuelve al menu**, no se cierra.

3.1 Diagrama de flujo

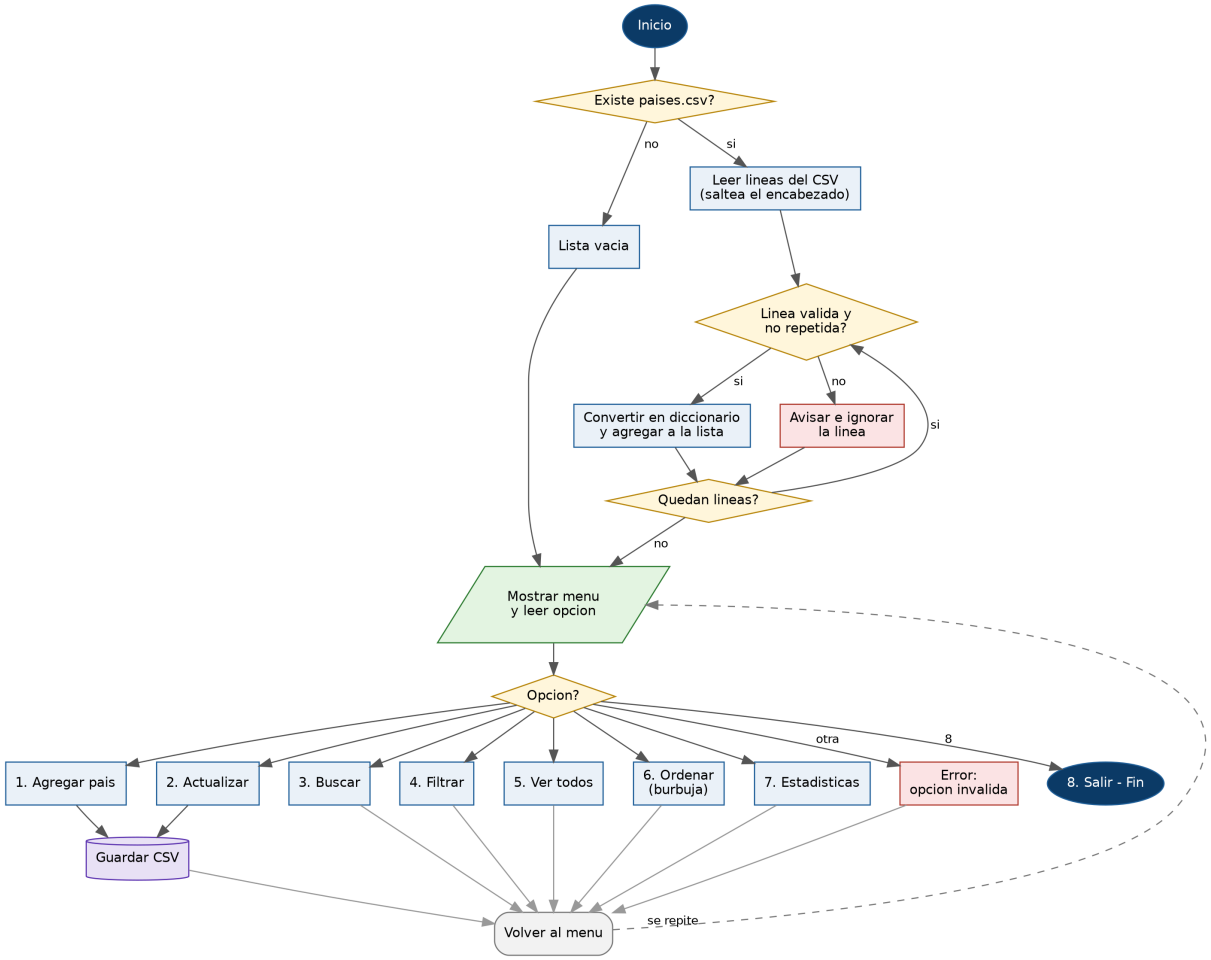


Figura 1: Flujo general del programa.

3.2 Capturas de ejecucion

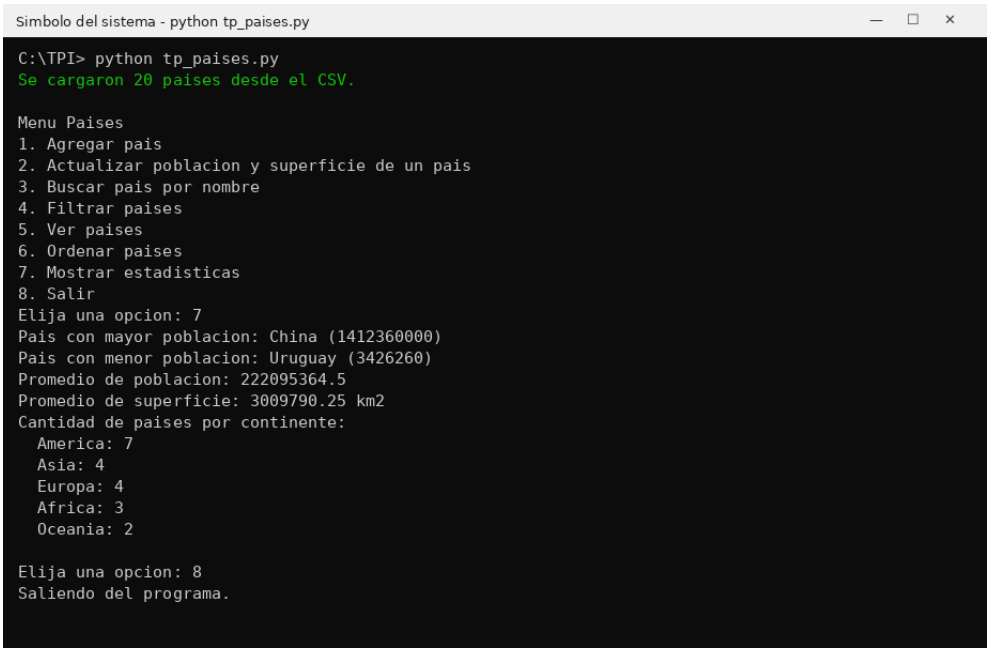


Figura 2: Inicio y estadisticas.

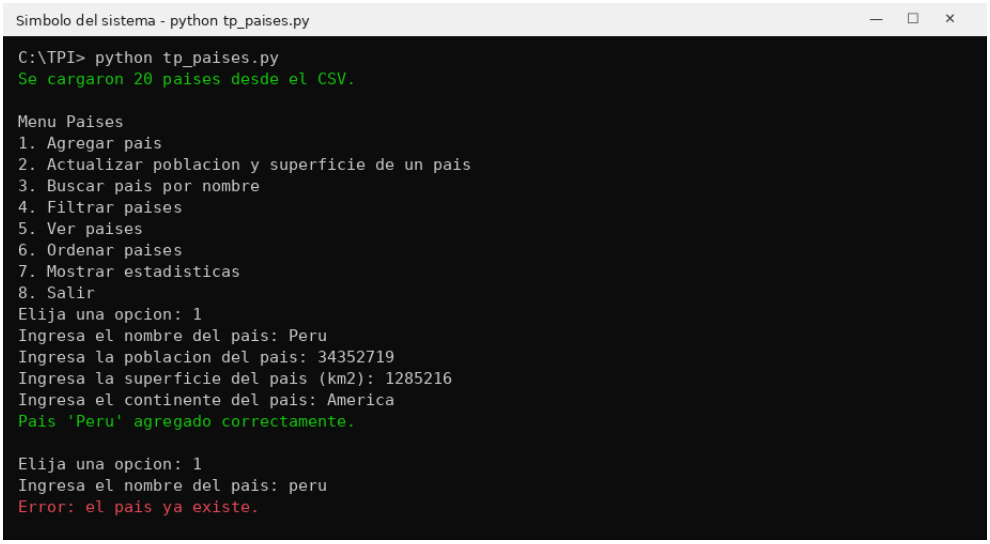


Figura 3: Agregar pais (y error si ya existe).



Figura 4: Actualizar poblacion y superficie.

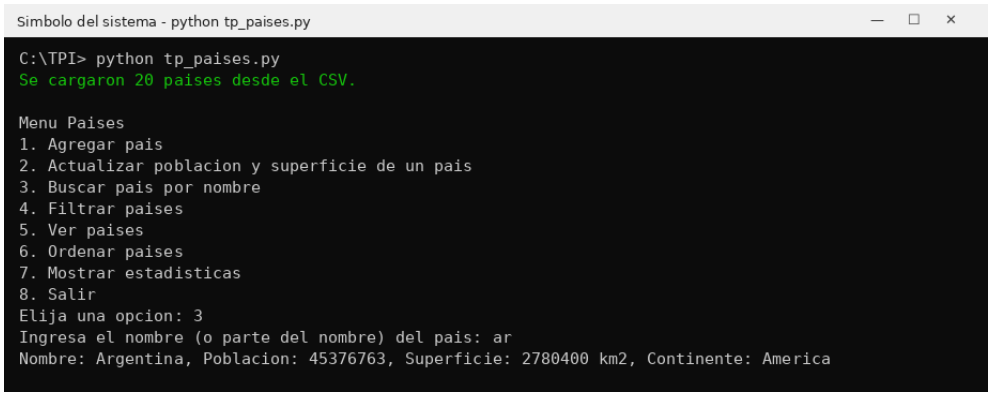


Figura 5: Busqueda parcial.

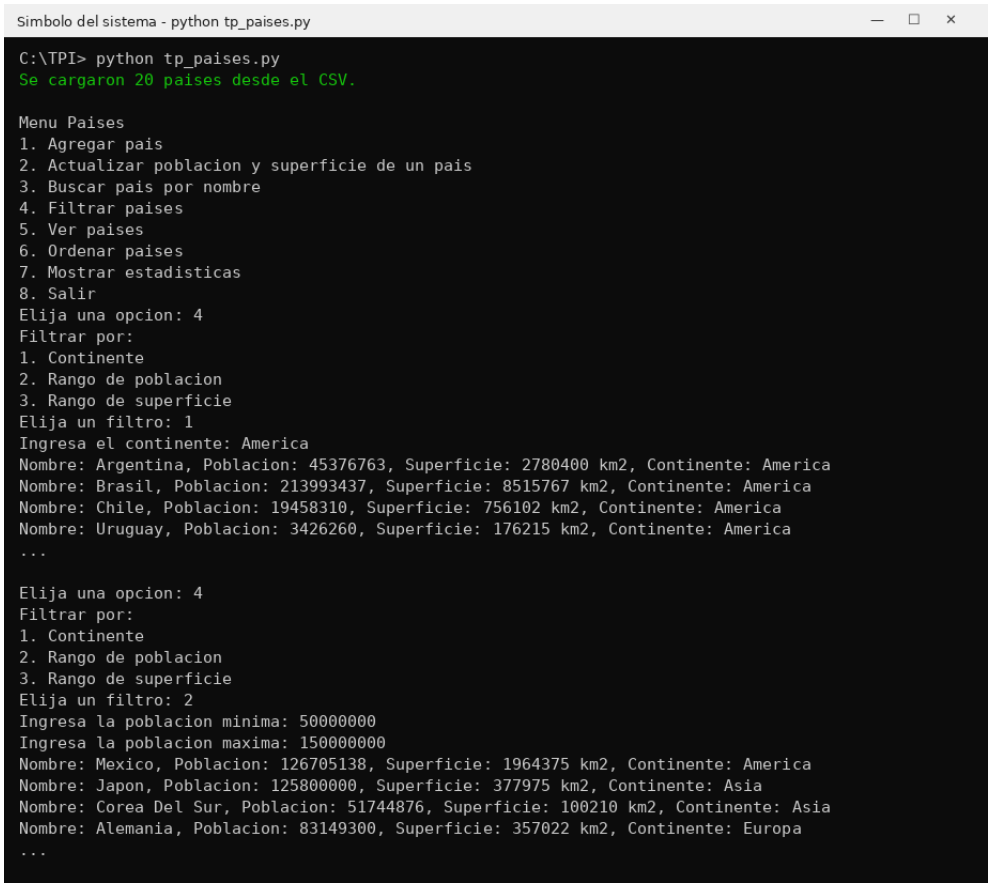


Figura 6: Filtro por continente y por rango de poblacion.

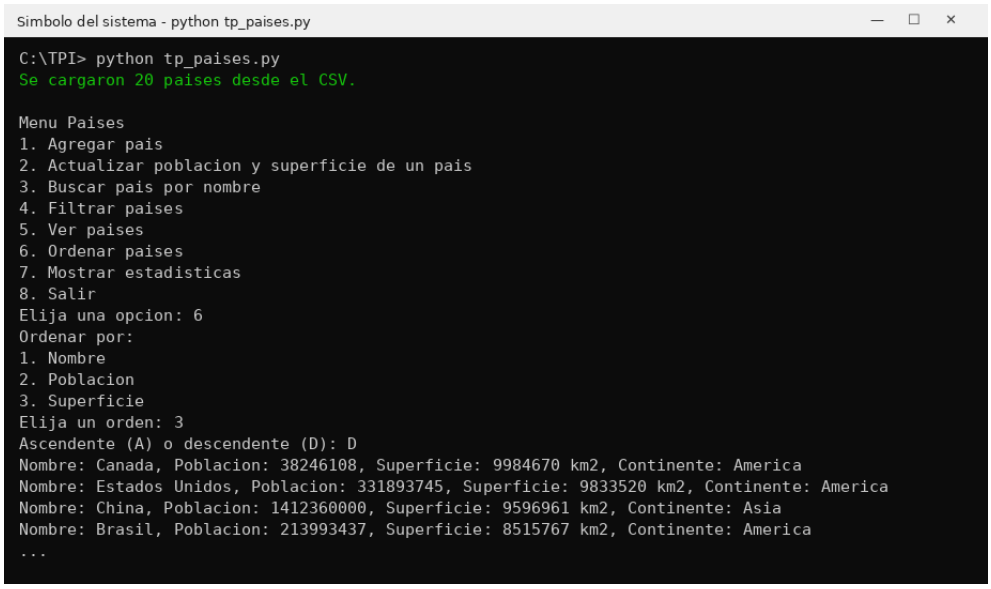


Figura 7: Orden por superficie, de mayor a menor.

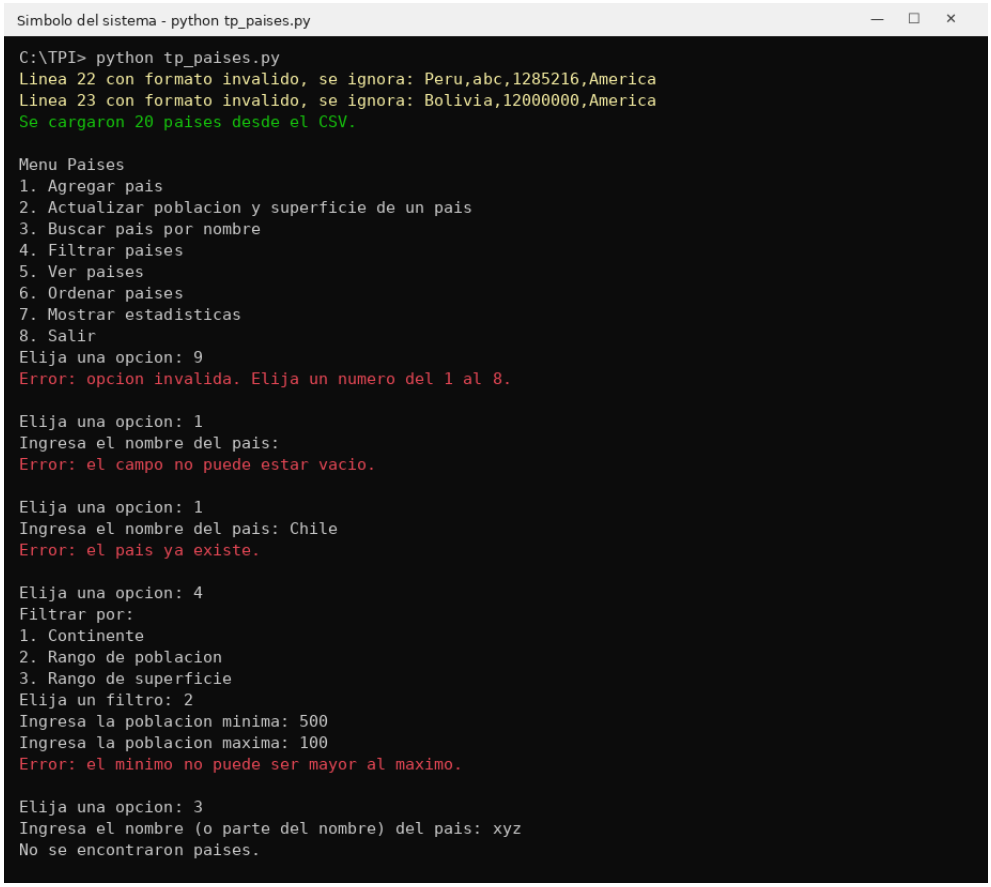


Figura 8: Errores y validaciones.

4. Dificultades y Conclusiones

4.1 Dificultades

- **Las tildes:** si en el CSV decia "America" con tilde y filtrabamos sin tilde, no aparecia nada. Lo arreglamos sacando las tildes del CSV y de lo que escribe el usuario.
- **Datos mal escritos:** si ponias letras donde iba un numero, el programa se cerraba. Agregamos validaciones antes de convertir.
- **El CSV no estaba:** si se borraba el archivo, el programa no arrancaba. Lo resolvimos con `try/except`.
- **El metodo burbuja:** nos costo entenderlo, lo probamos a mano con listas chicas hasta que salio.

4.2 Conclusiones

El trabajo nos sirvio para juntar casi todo lo que vimos en la materia en un solo programa. Nos dimos cuenta de que dividir el codigo en funciones hace todo mas facil de arreglar, y que gran parte de programar es pensar en lo que puede salir mal y hacer que el programa no se rompa.

5. Links del proyecto

Repositorio en GitHub: https://github.com/bauttigil-hue/tpi_programacion

Video demostrativo: https://youtu.be/8hB6iG72dzg